

Laboratorium – dokumentowe bazy danych: Couchbase

Temat: Couchbase, dokumenty JSON, indeksy, JOIN, UNNEST i analiza danych Northwind

Baza: Couchbase Community uruchomiony w Dockerze

Bucket: northwind

Scope: _default

Główne kolekcje: orders, orderdetails, customers, products, orders_nested

Imię i nazwisko: Marek Małek, Mateusz Lampert

Grupa: 4, piątek 15:00-16:30

Cel ćwiczenia

Po wykonaniu laboratorium student powinien umieć:

1. uruchomić i zweryfikować środowisko Couchbase,
2. poruszać się po panelu Couchbase i korzystać z Query Workbench,
3. rozumieć strukturę bucket → scope → collection,
4. wykonać podstawowe zapytania SQL++ / N1QL na dokumentach JSON,
5. wyjaśnić, dlaczego Couchbase wymaga indeksów do wykonywania zapytań,
6. odróżnić primary index od indeksu celowego,
7. wykonać JOIN między kolekcjami dokumentów,
8. porównać model niezagnieżdżony z modelem zagnieżdżonym,
9. użyć UNNEST do rozbijania tablicy zagnieżdżonej w dokumencie,
10. wykorzystać EXPLAIN do podstawowej interpretacji planu zapytania.

Ważne informacje

W tym laboratorium pracujemy na danych Northwind załadowanych do Couchbase.

Dane są dostępne w dwóch wariantach modelowania:

Model niezagnieżdżony

Dane są podzielone na osobne kolekcje:

- orders – zamówienia,
- orderdetails – pozycje zamówień,
- customers – klienci,
- products – produkty.

W tym wariancie, aby połączyć zamówienia z pozycjami zamówień, używamy JOIN.

Model zagnieżdżony

Dodatkowo przygotowana jest kolekcja:

- orders_nested.

W tej kolekcji jeden dokument odpowiada jednemu zamówieniu, a pozycje zamówienia są zapisane wewnątrz dokumentu w tablicy items. W tym wariancie do rozbicia pozycji zamówienia używamy UNNEST.

Jak korzystać ze ściąg

Do laboratorium dołączona jest ściąg Couchbase_SQLPP_sciaga.md. Korzystaj z niej jak z dokumentacji pomocniczej: sprawdzaj składnię JOIN, UNNEST, IFMISSINGORNULL, CREATE INDEX i EXPLAIN, ale nie kopiuj bezrefleksyjnie gotowych rozwiązań. Oceniane są również komentarze i interpretacja.

Sprawozdanie

Oddawane sprawozdanie powinno zawierać:

- kod zapytań,
- wyniki zapytań jako tabele albo zrzuty ekranu,
- krótkie komentarze interpretacyjne,
- odpowiedzi na pytania wskazane w zadaniach.

Format sprawozdania: PDF albo Markdown.

Kod SQL++ / N1QL formatuj jako bloki kodu.

Nie oddawaj samych zrzutów ekranu bez komentarza.

Punktacja

Zadanie	Temat	Punkty
0	Gotowość środowiska	0
1	Pierwsze poznanie Couchbase i danych	1
2	Indeksy: brak indeksu, primary index, secondary index	2
3	JOIN na kolekcjach dokumentów	2
4	Model niezagnieżdżony vs zagnieżdżony: JOIN vs UNNEST	2
5	Agregacja biznesowa	1
6	EXPLAIN i refleksja końcowa	2
Razem		10

0. Gotowość środowiska – 0 pkt

To zadanie nie jest punktowane, ale jest warunkiem rozpoczęcia pracy.

Wykonaj

1. Uruchom środowisko:

`docker compose --profile init up -d`

2. Sprawdź, czy działają kontenery:

`docker ps`

3. Wejdź do panelu Couchbase:

`http://localhost:8091`

4. Zaloguj się:

Login: `student`

Hasło: `student`

5. Sprawdź, czy widzisz bucket:

`northwind`

6. Wejdź do Query Workbench i uruchom:

`SELECT 1 AS test;`

Do sprawozdania

Nie trzeba dołączać pełnych logów. Wystarczy jedno zdanie:

Środowisko Couchbase zostało uruchomione, logowanie działa, bucket northwind jest widoczny, a Query Workbench wykonuje zapytania.

1. Pierwsze poznanie Couchbase i danych – 1 pkt

Cel

Poznaj podstawową strukturę danych w Couchbase i sprawdź, jakie kolekcje są dostępne.

Wykonaj

1. W panelu Couchbase za pomocą zakładek w menu bocznym np. ‘Buckets’, ‘Documents’, ‘Query’ zbadaj strukturę:

bucket → scope → collection

2. W Query Workbench policz liczbę dokumentów w kolekcjach:

- orders,
- orderdetails,
- customers,
- products,
- orders_nested.

W poprawnie zainicjalizowanym środowisku powinienes otrzymać orientacyjnie:

Kolekcja	Liczba dokumentów
orders	83?
orderdetails	215?
customers	9?
products	7?
orders_nested	83?

3. Podejrzyj kilka dokumentów z kolekcji orders.
4. Podejrzyj jeden dokument z kolekcji orders_nested.

Wskazówki

Przykład zapytania liczącego dokumenty:

```
SELECT COUNT(1) AS orders_count
FROM `northwind`._default.orders
WHERE OrderID IS NOT MISSING;
```

Przykład podejrzenia dokumentu:

```
SELECT o
FROM `northwind`._default.orders AS o
WHERE o.OrderID IS NOT MISSING
LIMIT 3;
```

W komentarzu napisz

- Co oznaczają pojęcia bucket, scope i collection?
- Ile dokumentów znajduje się w kolekcjach orders, orderdetails, customers, products i orders_nested?
- Czym różni się dokument z kolekcji orders od dokumentu z kolekcji orders_nested?

Rozwiązanie:

Przykładowe dokumenty z kolekcji orders:

```
SELECT o
FROM orders AS o
WHERE o.OrderID IS NOT MISSING
LIMIT 2;

[
  {
    "o": {
      "CustomerID": "VINET",
      "EmployeeID": 5,
      "Freight": 32.38,
      "OrderDate": {
        "$date": "1996-07-04T00:00:00Z"
      },
      "OrderID": 10248,
      "RequiredDate": {
        "$date": "1996-08-01T00:00:00Z"
      },
      "ShipAddress": "59 rue de l'Abbaye",
      "ShipCity": "Reims",
      "ShipCountry": "France",
      "ShipName": "Vins et alcools Chevalier",
      "ShipPostalCode": "51100",
      "ShipRegion": null,
      "ShipVia": 3,
      "ShippedDate": {
        "$date": "1996-07-16T00:00:00Z"
      },
      "_id": {
        "$oid": "63a060b9bb3b972d6f4e1fc6"
      }
    }
  },
  {
    "o": {
      "CustomerID": "TOMSP",
      "EmployeeID": 6,
      "Freight": 11.61,
      "OrderDate": {
        "$date": "1996-07-05T00:00:00Z"
      },
      "OrderID": 10249,
      "RequiredDate": {
        "$date": "1996-08-16T00:00:00Z"
      },
      "ShipAddress": "Luisenstr. 48",
      "ShipCity": "Münster",
      "ShipCountry": "Germany",
      "ShipName": "Toms Spezialitäten",
      "ShipPostalCode": "44087",
      "ShipRegion": null,
      "ShipVia": 1,
      "ShippedDate": {
        "$date": "1996-07-10T00:00:00Z"
      },
      "_id": {
        "$oid": "63a060b9bb3b972d6f4e1fc7"
      }
    }
  }
]
```

Przykładowy dokument z kolekcji orders_nested:

```
SELECT odn
FROM orders_nested AS odn
WHERE odn.OrderID IS NOT MISSING
LIMIT 1;

[
  {
    "odn": {
      "CustomerID": "VINET",
      "EmployeeID": 5,
      "OrderDate": {
        "$date": "1996-07-04T00:00:00Z"
      },
      "OrderID": 10248,
      "ShipCity": "Reims",
      "ShipCountry": "France",
      "ShipName": "Vins et alcools Chevalier",
      "items": [
        {
          "Discount": 0,
          "LineValue": 98,
          "ProductID": 42,
          "Quantity": 10,
          "UnitPrice": 9.8
        },
        {
          "Discount": 0,
          "LineValue": 168,
          "ProductID": 11,
          "Quantity": 12,
          "UnitPrice": 14
        },
        {
          "Discount": 0,
          "LineValue": 174,
          "ProductID": 72,
          "Quantity": 5,
          "UnitPrice": 34.8
        }
      ]
    },
    "type": "order_nested"
  }
]
```

Co oznaczają pojęcia bucket, scope i collection?

Bucket - najwyższy poziom w hierarchii organizacji danych, jest to odpowiednik całej bazy danych. W ramach bucketu może istnieć wiele różnych scope.

Scope - pośredni poziom grupowania kolekcji wewnątrz bucketa, odpowiednik przestrzeni nazw. Pozwala na logiczne oddzielenie od siebie zestawów kolekcji (np. sprzedaż vs. magazyn). W ramach scope może istnieć wiele różnych kolekcji.

Collection - zbiór dokumentów tego samego typu, jest to odpowiednik tabeli w SQL, natomiast kolekcje nie narzucają konkretnej struktury, reprezentują jedynie zbiór podobnych dokumentów. W ramach kolekcji może istnieć wiele dokumentów.

Ile dokumentów znajduje się w kolekcjach orders, orderdetails, customers, products i orders_nested?

Query Editor

< history (60/60) >

context

northwind

_default

1 SELECT COUNT(1) AS orders_count

2 FROM orders

3 WHERE OrderID IS NOT MISSING;

Execute

Explain

External Query Advisor

✓ success

just now | 8.9ms | 1 docs | 35 bytes

Results

Table

JSON

 Plan

1 [

2 {

3 "orders_count": 830

4 }

5]

Wizualizacja 1: Liczba dokumentów w kolekcji orders

Query Editor

< history (46/46) >

context

northwind

_default

1 SELECT COUNT(1) AS ordersdetails_count

2 FROM orderdetails

3 WHERE OrderID IS NOT MISSING;

Execute

Explain

External Query Advisor

✓ success

just now | 5.2ms | 1 docs | 43 bytes

Results

Table

JSON

 Plan

1 [

2 {

3 "ordersdetails_count": 2155

4 }

5]

Wizualizacja 2: Liczba dokumentów w kolekcji orderdetails

Query Editor

< history (57/57) >

context

northwind

_default

1 SELECT COUNT(1) AS customers_count

2 FROM customers

3 WHERE CustomerID IS NOT MISSING;

Execute

Explain

External Query Advisor

✓ success

just now | 13.2ms | 1 docs | 37 bytes

Results

Table

JSON

 Plan

1 ▾ [

2 ▾ {

3 "customers_count": 91

4 }

5]

Wizualizacja 3: Liczba dokumentów w kolekcji customers

Query Editor

< history (58/58) >

context

northwind

_default

1 SELECT COUNT(1) AS products_count

2 FROM products

3 WHERE ProductID IS NOT MISSING;

Execute

Explain

External Query Advisor

✓ success

just now | 7.1ms | 1 docs | 36 bytes

Results

Table

JSON

 Plan

1 ▾ [

2 ▾ {

3 "products_count": 77

4 }

5]

Wizualizacja 4: Liczba dokumentów w kolekcji products


```
1 SELECT COUNT(1) AS orders_nested_count
2 FROM orders_nested
3 WHERE OrderID IS NOT MISSING;
```

Execute Explain

External Query Advisor success just now | 6.6ms | 1 docs | 42 bytes

Results



Table JSON Plan

1 [

2 {

3 "orders_nested_count": 830

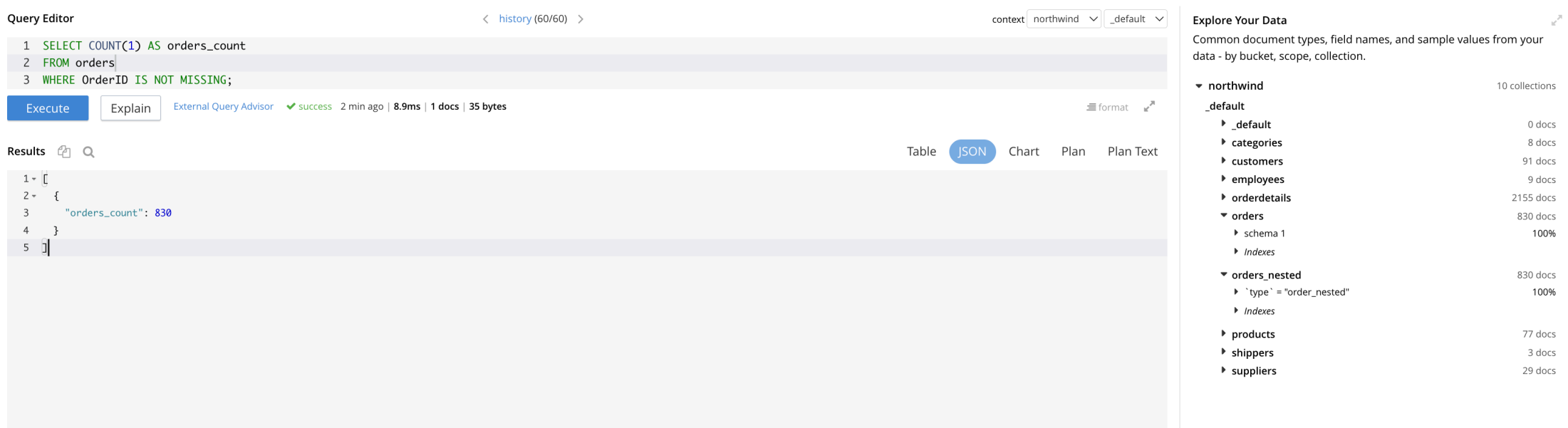
4 }

5]

Wizualizacja 5: Liczba dokumentów w kolekcji orders_nested

Kolekcja	Liczba dokumentów
orders	830
orderdetails	2155
customers	91
products	77
orders_nested	830

Dodatkowo, w Couchbase dostępne jest także podsumowanie danych, dzięki czemu mamy bezpośrednią informację o liczności poszczególnych kolekcji:



Wizualizacja 6: Podsumowanie kolekcji w panelu Couchbase

Czym różni się dokument z kolekcji orders od dokumentu z kolekcji orders_nested?

W kolekcji orders znajdują się tylko podstawowe informacje o zamówieniu, natomiast wszelkie szczegóły odnośnie kupowanych produktów znajdują się w kolekcji orderdetails (informacje o tym jakie produkty i w jakiej ilości były kupowane w ramach danego zamówienia). Kolekcja orders_nested zawiera te informacje bezpośrednio zagnieżdżone, to znaczy mamy do tych dancyh bezpośrednio, bez konieczności łączenia kolekcji (jak ma to miejsce w podejściu orders + orderdetails).

2. Indeksy: brak indeksu, primary index, secondary index – 2 pkt

Cel

Zobacz, że Couchbase nie wykonuje zapytań bez odpowiedniego indeksu. Następnie porównaj indeks główny z indeksem celowym.

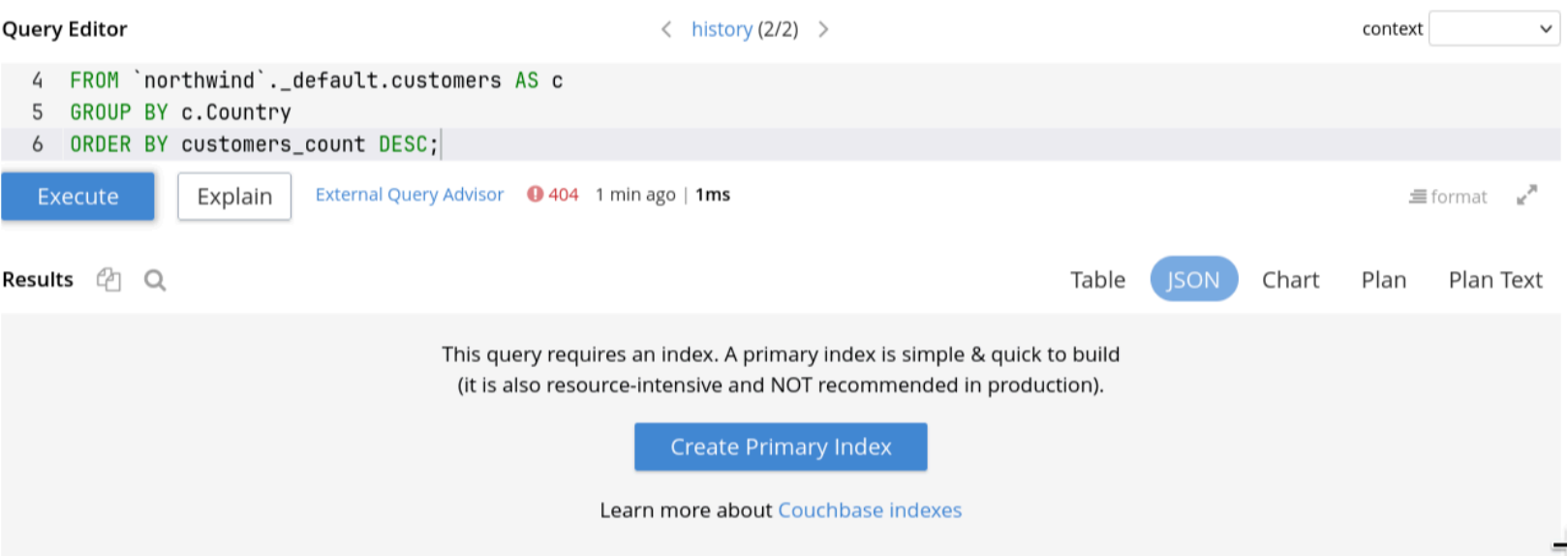
Część A – zapytanie bez indeksu

Wykonaj zapytanie:

```
SELECT
  c.Country,
  COUNT(1) AS customers_count
FROM `northwind`._default.customers AS c
GROUP BY c.Country
ORDER BY customers_count DESC;
```

Oczekiwane zachowanie

W świeżo uruchomionym środowisku zapytanie powinno zakończyć się błędem informującym o braku indeksu. Jeżeli zapytanie działa od razu, oznacza to najczęściej, że w środowisku pozostał wcześniej utworzony indeks.



Wizualizacja 7: Brak indeksu

Część B – primary index

Utwórz primary index:

```
CREATE PRIMARY INDEX idx_customers_primary
ON `northwind`.`_default.customers`;
```

Query Editor

< history (5/5) >

context

1 CREATE PRIMARY INDEX idx_customers_primary

2 ON `northwind`.`\_default.customers`;

Execute

Explain

External Query Advisor

success just now | 3s

format

Results

TableJSONChartPlanPlan Text

1 {

2 "results": []

3 }

Wizualizacja 8: Utworzony primary index

Powtórz zapytanie z części A.

Query Editor

< history (6/6) >

context

1 SELECT

2 c.Country,

3 COUNT(1) AS customers_count

Execute

Explain

External Query Advisor

success just now | 8.2ms | 21 docs | 1378 bytes

format

Results

TableJSONChartPlanPlan Text

1 [

2 {

3 "Country": "USA",

4 "customers_count": 13

5 },

6 {

7 "Country": "Germany",

8 "customers_count": 11

9 },

10 {

11 "Country": "France",

12 "customers_count": 11

13 },

Wizualizacja 9: Zapytanie z primary index

Część C – indeks celowy

Usuń primary index:

```
DROP INDEX idx_customers_primary
ON `northwind`.`_default.customers`;
```

Query Editor

< history (7/7) >

context

1 DROP INDEX idx_customers_primary

2 ON `northwind`.`\_default.customers`;

Execute

Explain

External Query Advisor

✔ success

just now | 28.5ms

format

Results

TableJSONChartPlanPlan Text

1 {

2 "results": []

3 }

Wizualizacja 10: Usunięty primary index

Utwórz indeks celowy na polu Country:

```
CREATE INDEX idx_customers_country
ON `northwind`.`_default.customers`(Country);
```

Query Editor

< history (8/8) >

context

1 CREATE INDEX idx_customers_country

2 ON `northwind`.`\_default.customers`(Country);

Execute

Explain

External Query Advisor

✔ success

just now | 3.1s

format

Results

TableJSONChartPlanPlan Text

1 {

2 "results": []

3 }

Wizualizacja 11: Utworzony indeks celowy

Wykonaj zapytanie z warunkiem indeksowym:

```
SELECT
  c.Country,
  COUNT(1) AS customers_count
FROM `northwind`.`_default.customers` AS c
WHERE c.Country IS NOT MISSING
GROUP BY c.Country
ORDER BY customers_count DESC;
```

Query Editor

< history (9/9) >

context

1 SELECT

2 c.Country,

3 COUNT(1) AS customers_count

Execute

Explain

External Query Advisor

✓ success

just now

|

5.2ms

|

21 docs

|

1378 bytes

format

Results

TableJSONChartPlanPlan Text

1

2

3

4

5

6

7

8

9

10

[

{

"Country": "USA",

"customers_count": 13

}

,

{

"Country": "France",

"customers_count": 11

}

,

{

Wizualizacja 12: Zapytanie z indeksem celowym

W komentarzu napisz

- Co się stało przy próbie wykonania zapytania bez indeksu?

Zapytanie zakończyło się błędem, ponieważ Couchbase wymaga indeksu do wykonania zapytania.

- Czym różni się primary index od indeksu celowego?

Primary index jest ogólnym indeksem, który pozwala na skanowanie wszystkich dokumentów w kolekcji, ale jest nieefektywny do większości zapytań. Indeks celowy jest zdefiniowany na konkretnych polach i umożliwia szybkie wyszukiwanie i agregację na tych polach.

- Dlaczego w zapytaniu po utworzeniu indeksu celowego dodano warunek WHERE c.Country IS NOT MISSING?

Warunek WHERE c.Country IS NOT MISSING jest potrzebny, aby zapytanie mogło skorzystać z indeksu celowego, który został utworzony na polu Country. Bez tego warunku Couchbase nie mógłby użyć indeksu i musiałby wykonać pełne skanowanie kolekcji, co jest nieefektywne.

- Dlaczego w środowisku produkcyjnym nie powinno się traktować primary index jako rozwiązania docelowego?

Primary index jest nieefektywny, ponieważ pozwala na skanowanie wszystkich dokumentów w kolekcji, co może prowadzić do długiego czasu odpowiedzi i dużego obciążenia bazy. W środowisku produkcyjnym powinno się tworzyć indeksy celowe na polach, które są często używane w zapytaniach, aby zapewnić szybkie i efektywne działanie aplikacji.

3. JOIN na kolekcjach dokumentów – 2 pkt

Cel

Zobacz, że Couchbase pozwala wykonywać JOIN podobny do SQL, ale pracuje na dokumentach JSON i kolekcjach, a nie na klasycznych tabelach relacyjnych.

Część A – zamówienia z nazwą klienta

Dla zamówień pokaż:

- OrderID,
- OrderDate,
- CustomerID,
- CompanyName.

Przed wykonaniem zapytania utwórz indeks potrzebny do łączenia z kolekcją customers:

```
CREATE INDEX idx_customers_customerid
ON `northwind`._default.customers(CustomerID);
```

Jeżeli indeks już istnieje, Couchbase zwróci komunikat o istniejącym indeksie. W takiej sytuacji przejdź do kolejnego kroku.

Następnie przygotuj zapytanie łączące orders z customers.

Część B – zamówienia i pozycje zamówień

Dla pozycji zamówień pokaż:

- OrderID,
- CustomerID,
- ProductID,
- UnitPrice,
- Quantity,
- Discount.

Wykorzystaj kolekcje:

- orders,
- orderdetails.

Uwaga: indeksy idx_orders_orderid oraz idx_orderdetails_orderid zostały utworzone automatycznie podczas inicjalizacji środowiska. Nie trzeba ich zakładać ręcznie. Jeżeli mimo to spróbujesz je utworzyć, Couchbase poinformuje, że indeks już istnieje – to nie jest błąd.

Rozwiązanie:

```
SELECT o.OrderID,
       o.CustomerID,
       od.ProductID,
       od.UnitPrice,
       od.Quantity,
       od.Discount
FROM orders AS o
      JOIN orderdetails AS od ON o.OrderID = od.OrderID
WHERE o.OrderID IS NOT MISSING
      AND od.OrderID IS NOT MISSING
```

Query Editor

< history (70/70) >

context northwind _default

```
1 SELECT o.OrderID,
2       o.CustomerID,
3       od.ProductID,
4       od.UnitPrice,
5       od.Quantity,
6       od.Discount
7 FROM orders AS o
8      JOIN orderdetails AS od ON o.OrderID = od.OrderID
9 WHERE o.OrderID IS NOT MISSING
10      AND od.OrderID IS NOT MISSING
11 LIMIT 10;
```

Execute

Explain

External Query Advisor

success

just now

16.5ms

format

10 docs

1667 bytes

Results

Table

JSON

Chart

Plan

Plan Text

```
1 [
2   {
3     "CustomerID": "VINET",
4     "Discount": 0,
5     "OrderID": 10248,
6     "ProductID": 11,
7     "Quantity": 12,
8     "UnitPrice": 14
9   },
10  {
11    "CustomerID": "VINET",
12    "Discount": 0,
13    "OrderID": 10248,
14    "ProductID": 72,
15    "Quantity": 5,
16    "UnitPrice": 34.8
17  },
18  {
19    "CustomerID": "VINET",
20    "Discount": 0,
21    "OrderID": 10248,
22    "ProductID": 42,
23    "Quantity": 10,
24    "UnitPrice": 9.8
25  },
26  {
27    "CustomerID": "TOMSP",
28    "Discount": 0,
29    "OrderID": 10249,
30    "ProductID": 51,
31    "Quantity": 40,
```

Wizualizacja 13: Wynik zapytania JOIN dla zamówień i pozycji zamówień

Część C – wartość zamówienia

Policz wartość zamówienia według wzoru:

wartość pozycji = UnitPrice * Quantity * (1 - Discount)

Brak rabatu traktuj jako 0.

Wskazówka: w dokumentach JSON pole Discount może nie istnieć albo mieć wartość null. W Couchbase do obsługi takich sytuacji służy funkcja IFMISSINGORNULL(od.Discount, 0), która zwraca 0, jeżeli pole nie istnieje lub jest puste.

Dla każdego zamówienia oblicz:

- OrderID,
- CustomerID,
- order_value,
- liczbę pozycji zamówienia.

Pokaż 10 zamówień o najwyższej wartości.

Rozwiązanie:

```
SELECT o.OrderID,
       o.CustomerID,
       SUM(od.UnitPrice * od.Quantity * (1 - IFMISSINGORNULL(od.Discount, 0))) AS order_value
FROM orders AS o
     JOIN orderdetails AS od ON o.OrderID = od.OrderID
WHERE o.OrderID IS NOT MISSING
     AND od.OrderID IS NOT MISSING
GROUP BY o.OrderID,
         o.CustomerID
LIMIT 10;
```

```
[
  {
    "CustomerID": "QUICK",
    "OrderID": 10938,
    "order_value": 2731.875
  },
  {
    "CustomerID": "TRADH",
    "OrderID": 10830,
    "order_value": 1974
  },
  {
    "CustomerID": "REGGC",
    "OrderID": 10942,
    "order_value": 560
  },
  {
    "CustomerID": "LILAS",
    "OrderID": 10283,
    "order_value": 1414.8000000000002
  },
  {
    "CustomerID": "OTTIK",
    "OrderID": 10508,
    "order_value": 240
  },
  {
    "CustomerID": "OCEAN",
    "OrderID": 10409,
    "order_value": 319.20000000000005
  },
  {
    "CustomerID": "SANTG",
    "OrderID": 10909,
    "order_value": 670
  },
  {
    "CustomerID": "SAVEA",
    "OrderID": 10678,
    "order_value": 5256.5
  },
  {
    "CustomerID": "REGGC",
    "OrderID": 10908,
    "order_value": 663.0999994799495
  },
  {
    "CustomerID": "RANCH",
    "OrderID": 10828,
    "order_value": 932
  }
]
```


Query Editor

< history (73/73) >

context northwind _default

1 SELECT o.OrderID,

2 o.CustomerID,

3 SUM(od.UnitPrice * od.Quantity * (1 - IFMISSINGORNULL(od.Discount,

4 FROM orders AS o

5 JOIN orderdetails AS od ON o.OrderID = od.OrderID

6 WHERE o.OrderID IS NOT MISSING

7 AND od.OrderID IS NOT MISSING

8 GROUP BY o.OrderID,

9 o.CustomerID

10 LIMIT 10;

Execute

Explain

External Query Advisor

success just now | 231.7ms

format

10 docs | 987 bytes

Results

Table JSON Chart Plan Plan Text

1 [

2 {

3 "CustomerID": "QUICK",

4 "OrderID": 10273,

5 "order_value": 2037.2799984335898

6 },

7 {

8 "CustomerID": "SPECB",

9 "OrderID": 10964,

10 "order_value": 2052.5

11 },

12 {

13 "CustomerID": "MAGAA",

14 "OrderID": 10404,

15 "order_value": 1591.2499987520278

16 },

17 {

18 "CustomerID": "MAISD",

19 "OrderID": 10892,

20 "order_value": 2089.9999983608723

21 },

22 {

23 "CustomerID": "QUICK",

24 "OrderID": 10996,

25 "order_value": 560

26 },

27 {

28 "CustomerID": "OTTIK",

29 "OrderID": 11020,

30 "order_value": 632.3999955654144

31 },

32 {

Wizualizacja 14: Wynik zapytania agregującego wartość zamówień

W komentarzu napisz

- Czy JOIN w Couchbase przypomina składnię znaną z SQL?
- Czym różni się takie łączenie od relacji w klasycznej bazie relacyjnej (np. czy baza wymusza klucze obce i spójność relacji tak jak w typowym modelu relacyjnym)?
- Dlaczego indeks po stronie dołączanej kolekcji jest ważny?
- Czy największe zamówienia mają zawsze największą liczbę pozycji?

Czy JOIN w Couchbase przypomina składnię znaną z SQL?

Składnia JOIN z Couchbase jest de facto identyczna w porównaniu do składni znanej z SQL.

Czym różni się takie łączenie od relacji w klasycznej bazie relacyjnej (np. czy baza wymusza klucze obce i spójność relacji tak jak w typowym modelu relacyjnym)?

W klasycznej bazie relacyjnej baza danych wymusza klucze obce oraz ich integralność. W przypadku Couchbase’a baza nie przechowuje żadnych informacji o relacjach pomiędzy kolekcjami i nie waliduje spójności przy zapisie. Odpowiedzialność za integralność danych leży całkowicie po stronie aplikacji i programisty.

Dlaczego indeks po stronie dołączanej kolekcji jest ważny?

W przypadku braku indeksu konieczne pełne przeskanowanie kolekcji, czyli w przypadku sprawdzenie każdego dokumentu w tabeli orderdetails dla każdego wiersza z orders. Indeks na kolumnie, po której łączymy kolekcje pozwala silnikowi bezpośrednio zlokalizować pasujące dokumenty bez przeglądania całej kolekcji.

W przypadku próby połączenia kolekcji po kluczu, na który nie jest założony indeks, zapytanie kończy się błędem:

```
[
  {
    "code": 4330,
    "msg": "No index available for ANSI join term od",
    "query": "SELECT o.OrderID,\n          o.ProductID,\n          SUM(od.UnitPrice * od.Quantity * (1 - IFMISSINGORNULL(od.Discount, 0))) AS order_value,\n          COUNT(*) AS\npositions\nFROM orders AS o\n  JOIN orderdetails AS od ON o.OrderID = od.ProductID\nWHERE o.OrderID IS NOT MISSING\n  AND od.ProductID IS NOT MISSING\nGROUP BY\n  o.OrderID,\n  o.ProductID\nORDER BY order_value DESC, positions DESC\nLIMIT 10;"
  }
]
```

Czy największe zamówienia mają zawsze największą liczbę pozycji?

W celu zweryfikowania tego stwierdzenia, do poprzedniego zapytania dodane zostało zliczanie liczby pozycji w zamówieniu:

```
SELECT o.OrderID,
       o.CustomerID,
       SUM(od.UnitPrice * od.Quantity * (1 - IFMISSINGORNULL(od.Discount, 0))) AS order_value,
       COUNT(*) AS positions
FROM orders AS o
  JOIN orderdetails AS od ON o.OrderID = od.OrderID
WHERE o.OrderID IS NOT MISSING
  AND od.OrderID IS NOT MISSING
GROUP BY o.OrderID,
         o.CustomerID
ORDER BY order_value DESC, positions DESC;
```

```
[
  {
    "CustomerID": "QUICK",
    "OrderID": 10865,
    "order_value": 16387.49998714775,
    "positions": 2
  },
  {
    "CustomerID": "HANAR",
    "OrderID": 10981,
    "order_value": 15810,
    "positions": 1
  },
  {
    "CustomerID": "SAVEA",
    "OrderID": 11030,
    "order_value": 12615.05,
    "positions": 4
  },
  {
    "CustomerID": "RATTC",
    "OrderID": 10889,
    "order_value": 11380,
    "positions": 2
  },
  {
    "CustomerID": "SIMOB",
    "OrderID": 10417,
    "order_value": 11188.4,
    "positions": 4
  },
  {
    "CustomerID": "KOENE",
    "OrderID": 10817,
    "order_value": 10952.844978627563,
    "positions": 4
  },
  {
    "CustomerID": "HUNGO",
    "OrderID": 10897,
    "order_value": 10835.240000000002,
    "positions": 2
  },
  {
    "CustomerID": "RATTC",
    "OrderID": 10479,
    "order_value": 10495.6,
    "positions": 4
  },
  {
    "CustomerID": "QUICK",
    "OrderID": 10540,
    "order_value": 10191.7,
    "positions": 4
  },
  {
    "CustomerID": "QUICK",
    "OrderID": 10691,
    "order_value": 10164.8,
    "positions": 5
  }
]
```

Zgodnie z otrzymanymi wynikami (top 10 rezultatów w zadanej kolejności), największe zamówienia niekoniecznie zawsze mają największą liczbę pozycji. Zamówienie o największej wartości ma wyłącznie 2 pozycje, drugie w kolejności zamówienie ma 1 pozycję, gdzie np. zamówienie o 10. w kolejności wartości ma aż 5 pozycji.

4. Model niezagnieżdżony vs zagnieżdżony: JOIN vs UNNEST – 2 pkt

Cel

Porównaj dwa sposoby modelowania tych samych danych:

- 1. model niezagnieżdżony: orders + orderdetails,
- 2. model zagnieżdżony: orders_nested, gdzie pozycje zamówienia są tablicą items.

Część A – obejrzyj dokument zagnieżdżony

Podejrzyj dokument zamówienia o numerze 10248 z kolekcji orders_nested.

Zwróć uwagę na pole:

items

Rozwiązanie:

```
select *
from orders_nested
where OrderID = 10248;

[
  {
    "orders_nested": {
      "CustomerID": "VINET",
      "EmployeeID": 5,
      "OrderDate": {
        "$date": "1996-07-04T00:00:00Z"
      },
      "OrderID": 10248,
      "ShipCity": "Reims",
      "ShipCountry": "France",
      "ShipName": "Vins et alcools Chevalier",
      "items": [
        {
          "Discount": 0,
          "LineValue": 98,
          "ProductID": 42,
          "Quantity": 10,
          "UnitPrice": 9.8
        },
        {
          "Discount": 0,
          "LineValue": 168,
          "ProductID": 11,
          "Quantity": 12,
          "UnitPrice": 14
        },
        {
          "Discount": 0,
          "LineValue": 174,
          "ProductID": 72,
          "Quantity": 5,
          "UnitPrice": 34.8
        }
      ]
    },
    "type": "order_nested"
  }
]
```

Część B – rozbij tablicę items przez UNNEST

Dla zamówienia 10248 pokaż wszystkie pozycje zamówienia z tablicy items.

Wynik powinien zawierać:

- OrderID,
- CustomerID,
- ProductID,
- UnitPrice,
- Quantity,
- Discount,
- LineValue.

Wskazówka składniowa

UNNEST rozbija tablicę zagnieżdżoną w dokumencie na osobne rekordy. Każdy element tablicy staje się oddzielnym wierszem w wyniku:

```
SELECT
    n.OrderID,
    item.ProductID
FROM `northwind`.`_default.orders_nested` AS n
UNNEST n.items AS item
WHERE n.OrderID = 10248;
```

Rozbuduj to zapytanie o pozostałe kolumny wymienione powyżej.

Rozwiązanie:

```
SELECT odn.OrderID,
       odn.CustomerID,
       item.ProductID,
       item.UnitPrice,
       item.Quantity,
       item.Discount,
       item.LineValue
FROM orders_nested AS odn
UNNEST odn.items AS item
WHERE odn.OrderID = 10248;

[
  {
    "CustomerID": "VINET",
    "Discount": 0,
    "LineValue": 98,
    "OrderID": 10248,
    "ProductID": 42,
    "Quantity": 10,
    "UnitPrice": 9.8
  },
  {
    "CustomerID": "VINET",
    "Discount": 0,
    "LineValue": 168,
    "OrderID": 10248,
    "ProductID": 11,
    "Quantity": 12,
    "UnitPrice": 14
  },
  {
    "CustomerID": "VINET",
    "Discount": 0,
    "LineValue": 174,
    "OrderID": 10248,
    "ProductID": 72,
    "Quantity": 5,
    "UnitPrice": 34.8
  }
]
```

Część C – policz wartość zamówień z modelu zagnieżdżonego

Na kolekcji orders_nested policz:

- OrderID,
- CustomerID,
- order_value,
- liczbę pozycji.

Użyj UNNEST.

Pokaż 10 zamówień o najwyższej wartości.

```
SELECT n.OrderID,
       n.CustomerID,
       SUM(item.UnitPrice * item.Quantity * (1 - IFMISSINGORNUL(item.Discount, 0))) AS order_value,
       COUNT(1) AS positions
FROM orders_nested AS n
UNNEST n.items AS item
WHERE n.OrderID IS NOT MISSING
GROUP BY n.OrderID,
         n.CustomerID
ORDER BY order_value DESC
LIMIT 10;
```

```
[
  {
    "CustomerID": "QUICK",
    "OrderID": 10865,
    "order_value": 16387.49998714775,
    "positions": 2
  },
  {
    "CustomerID": "HANAR",
    "OrderID": 10981,
    "order_value": 15810,
    "positions": 1
  },
  {
    "CustomerID": "SAVEA",
    "OrderID": 11030,
    "order_value": 12615.05,
    "positions": 4
  },
  {
    "CustomerID": "RATTC",
    "OrderID": 10889,
    "order_value": 11380,
    "positions": 2
  },
  {
    "CustomerID": "SIMOB",
    "OrderID": 10417,
    "order_value": 11188.4,
    "positions": 4
  },
  {
    "CustomerID": "KOENE",
    "OrderID": 10817,
    "order_value": 10952.844978627563,
    "positions": 4
  },
  {
    "CustomerID": "HUNGO",
    "OrderID": 10897,
    "order_value": 10835.240000000002,
    "positions": 2
  },
  {
    "CustomerID": "RATTC",
    "OrderID": 10479,
    "order_value": 10495.6,
    "positions": 4
  },
  {
    "CustomerID": "QUICK",
    "OrderID": 10540,
    "order_value": 10191.7,
    "positions": 4
  },
  {
    "CustomerID": "QUICK",
    "OrderID": 10691,
    "order_value": 10164.8,
    "positions": 5
  }
]
```


Wynik tego zapytania jest pusty, co oznacza, że oba podejścia dają ten sam wynik biznesowy:

```
{
  "results": []
}
```

W komentarzu napisz

- Na czym polega różnica między JOIN i UNNEST?
- Dlaczego w modelu zagnieżdżonym nie trzeba łączyć orders z orderdetails?
- Czy oba podejścia dają ten sam wynik biznesowy?
- Kiedy zagnieżdżanie pozycji zamówienia w dokumencie może być wygodne?
- Kiedy lepiej zostawić dane w osobnych kolekcjach?

Na czym polega różnica między JOIN i UNNEST?
JOIN łączy dwa osobne dokumenty z dwóch kolekcji na podstawie wspólnego klucza. UNNEST rozpakowuje tablicę wewnątrz jednego dokumentu i traktuje każdy jej element jako osobny wiersz.

Dlaczego w modelu zagnieżdżonym nie trzeba łączyć orders z orderdetails?
--

Ponieważ w modelu zagnieżdżonym, wszystkie dane z orderdetails są zawarte w kolekcji orders_nested pod kluczem items - dzięki temu mamy do nich bezpośredni dostęp (są one częścią dokumentu), a wszystko co potrzebne do wyliczenia wartości zamówienia znajduje się w jednym miejscu.

Czy oba podejścia dają ten sam wynik biznesowy?

Oba podejścia dają ten sam wynik biznesowy

Kiedy zagnieżdżanie pozycji zamówienia w dokumencie może być wygodne?

Zagnieżdżanie jest wygodne, gdy dane są silnie powiązane i zawsze odczytywane razem. Dzięki temu możemy uniknąć wielu kosztowych operacji JOIN, ponieważ mamy bezpośredni dostęp do zagnieżdżonych danych. Przykładowo, pozycje zamówienia oraz szczegóły dotyczące zamówienia rzadko mają sens bez kontekstu samego zamówienia.

Kiedy lepiej zostawić dane w osobnych kolekcjach?

Osobne kolekcje sprawdzą się lepiej w sytuacji, gdy odwołujemy się do danych z wielu miejsc - przykładowo, produkt (identyfikowany przez ProductID) pojawia się w wielu zamówieniach i nie ma sensu duplikowanie danych każdego produktu za każdym razem, gdy jest on zagnieżdżany. Dla danych, dla których potrzebujemy niezależnych zapytań (dane nie są ściśle powiązane z innym dokumentem i ma sa odpytywanie tej kolekcji w izolacji), rozdzielenie danych na osobne kolekcje także może okazać się wygodniejsze. Dzięki rozdzieleniu zagnieżdżonych danych na osobne kolekcje możemy także uniknąć duplikacji danych, co pomaga także w aktualizacji danych (np. zmiana nazwy produktu w jednym miejscu, a nie w wielu dokumentach).

5. Agregacja biznesowa – 1 pkt

Cel

Wykonaj prostą analizę biznesową na danych dokumentowych.

Wybierz jeden wariant

Do zaliczenia zadania wybierz jeden wariant. Jeżeli skończysz wcześniej, wykonaj drugi wariant jako ćwiczenie dodatkowe.

Wariant A – top 10 produktów po wartości sprzedaży

Dla produktów policz:

- ProductID,
- ProductName,
- łączną liczbę sprzedanych sztuk,
- łączną wartość sprzedaży.

Wykorzystaj kolekcje:

- orderdetails,
- products.

Przed zapytaniem może być potrzebny indeks:

```
CREATE INDEX idx_products_productid
ON `northwind`.`_default.products(ProductID);
```

```
CREATE INDEX idx_orderdetails_productid
ON `northwind`.`_default.orderdetails(ProductID);
```

Jeżeli indeks już istnieje, Couchbase zwróci komunikat o istniejącym indeksie — to nie jest błąd.

Rozwiązanie

Dodanie indeksu

Query Editor

< history (6/6) >

context

▼

1 CREATE INDEX idx_products_productid

2 ON `northwind`.`_default.products(ProductID);

3

Execute

Explain

External Query Advisor

✔ success just now

≡ format

↗

Results

📄

🔍

Table

JSON

ChartPlanPlan Text

1 [

2 {

3 "_sequence_num": 1,

4 "_sequence_query": "CREATE INDEX idx_products_productid\nON `northwind`.`_default.products(ProductID);",

5 "_sequence_query_status": "success",

6 "_sequence_result": {

7 "results": []

8 }

9 },

10 {

11 "_sequence_num": 2,

12 "_sequence_query": "\n\nCREATE INDEX idx_orderdetails_productid\nON `northwind`.`_default.orderdetails(ProductID);",

13 "_sequence_query_status": "success",

14 "_sequence_result": {

15 "results": []

16 }

17 }

18]

Wizualizacja 15: Dodanie indeksu

Zapytanie

```
select p.ProductID,
       p.ProductName,
       count(*)
       sum(o.UnitPrice * o.Quantity * (1 - ifmissingornull(o.Discount, 0))) as TotalUnitsSold,
       sum(o.UnitPrice * o.Quantity * (1 - ifmissingornull(o.Discount, 0))) as TotalOrderValue
from `northwind`.`_default.products` as p
     join `northwind`.`_default.orderdetails` as o
       on p.ProductID = o.ProductID
where p.ProductID is not missing
     and o.ProductID is not missing
group by p.ProductID, p.ProductName
order by TotalOrderValue desc
limit 10;
```

Wynik:

Query Editor

< history (25/25) >

context

1 select p.ProductID,

2 p.ProductName,

3 count(*)

as TotalUnitsSold,

Execute

Explain

External Query Advisor

✓ success

just now | 45.8ms | 10 docs | 1520 bytes

format

Results

TableJSONChartPlanPlan Text

1 [

2 {

3 "ProductID": 38,

4 "ProductName": "Côte de Blaye",

5 "TotalOrderValue": 141396.73490344844,

6 "TotalUnitsSold": 24

7 },

8 {

9 "ProductID": 29,

10 "ProductName": "Thüringer Rostbratwurst",

11 "TotalOrderValue": 80368.67197451406,

12 "TotalUnitsSold": 32

13 },

14 {

15 "ProductID": 59,

16 "ProductName": "Raclette Courdavault",

17 "TotalOrderValue": 71155.69990943,

18 "TotalUnitsSold": 54

19 },

20 {

21 "ProductID": 62,

22 "ProductName": "Tarte au sucre",

23 "TotalOrderValue": 47234.969954013824,

24 "TotalUnitsSold": 48

25 },

26 {

Wizualizacja 16: Wynik zapytania

Wariant B – top 10 klientów po wartości zakupów

Dla klientów policz:

- CustomerID,
- CompanyName,
- liczbę zamówień,
- łączną wartość zakupów.

Wykorzystaj kolekcje:

- orders,
- orderdetails,
- customers.

Rozwiązanie

Dodanie indeksu

```
CREATE INDEX idx_orders_customerid
ON `northwind`._default.orders(CustomerID);
```

```
CREATE INDEX idx_customers_customerid
ON `northwind`._default.customers(CustomerID);
```

Query Editor

< history (51/51) >

context

▼

1 CREATE INDEX idx_orders_customerid

2 ON `northwind`._default.orders(CustomerID);

3

Execute

Explain

External Query Advisor

✔ success just now

format

Results

Table

JSON

Chart

Plan

Plan Text

1 [

2 {

3 "_sequence_num": 1,

4 "_sequence_query": "CREATE INDEX idx_orders_customerid\nON `northwind`._default.orders(CustomerID);",

5 "_sequence_query_status": "success",

6 "_sequence_result": {

7 "results": []

8 }

9 },

10 {

11 "_sequence_num": 2,

12 "_sequence_query": "\n\nCREATE INDEX idx_customers_customerid\nON `northwind`._default.customers(CustomerID);",

13 "_sequence_query_status": "success",

14 "_sequence_result": {

15 "results": []

16 }

17 }

18]

Wizualizacja 17: Dodanie indeksu

Zapytanie

```
select c.CustomerID,
       c.CompanyName,
       count(distinct o.OrderID) as TotalOrderCount,
       sum(od.UnitPrice * od.Quantity * (1 - ifmissingornull(od.Discount, 0))) TotalOrderValue
from `northwind`.`_default.customers` as c
     join `northwind`.`_default.orders` as o
       on o.CustomerID = c.CustomerID
     join `northwind`.`_default.orderdetails` as od
       on od.OrderID = o.OrderID
group by c.CustomerID, c.CompanyName
order by TotalOrderValue desc
limit 10;
```

Wynik:

Query Editor

< history (37/37) >

context

1 select c.CustomerID, c.CompanyName,

2 count(distinct o.OrderID) as TotalOrderCount,

3 sum(od.UnitPrice * od.Quantity * (1 - ifmissingornull(od.Discount, 0))) TotalOrderValue

Execute

Explain

External Query Advisor

success just now | 177.4ms | 10 docs | 1589 bytes

format

Results

TableJSONChartPlanPlan Text

1 [

2 {

3 "CompanyName": "QUICK-Stop",

4 "CustomerID": "QUICK",

5 "TotalOrderCount": 28,

6 "TotalOrderValue": 110277.3048835089

7 },

8 {

9 "CompanyName": "Ernst Handel",

10 "CustomerID": "ERNSH",

11 "TotalOrderCount": 30,

12 "TotalOrderValue": 104874.97835350106

13 },

14 {

15 "CompanyName": "Save-a-lot Markets",

16 "CustomerID": "SAVEA",

17 "TotalOrderCount": 31,

18 "TotalOrderValue": 104361.94986812411

19 },

20 {

21 "CompanyName": "Rattlesnake Canyon Grocery",

22 "CustomerID": "RATTG",

23 "TotalOrderCount": 18,

24 "TotalOrderValue": 51097.800487737826

25 },

26]

Wizualizacja 18: Wynik zapytania

W komentarzu napisz

- Który produkt albo klient ma najwyższą wartość sprzedaży?

A:

```
{
  "ProductID": 38,
  "ProductName": "Côte de Blaye",
  "TotalOrderValue": 141396.73490344844,
  "TotalUnitsSold": 24
}
```

B:

```
{
  "CompanyName": "QUICK-Stop",
  "CustomerID": "QUICK",
  "TotalOrderCount": 28,
  "TotalOrderValue": 110277.3048835089
}
```

- Czy wynik jest łatwy do biznesowej interpretacji?

Tak, wynik jest łatwy do interpretacji, ponieważ pokazuje konkretne produkty i klientów wraz z ich łączną wartością sprzedaży i liczbą sprzedanych jednostek, co pozwala na szybkie zidentyfikowanie najbardziej wartościowych produktów i klientów.

- Czy zapytanie bardziej przypomina klasyczny SQL/BI, czy pracę z dokumentami JSON?

Zapytanie bardziej przypomina klasyczny SQL/BI, ponieważ wykonuje typowe operacje agregacji, grupowania i sortowania, ale używa też funkcji specyficznych dla Couchbase, takich jak IFMISSINGORNULL, aby radzić sobie z potencjalnie brakującymi danymi w dokumentach JSON.

6. EXPLAIN i refleksja końcowa – 2 pkt

Cel

Nie wystarczy wiedzieć, że zapytanie działa. Trzeba jeszcze rozumieć, w jaki sposób baza je wykonuje.

Część A – plan dla zapytania z JOIN

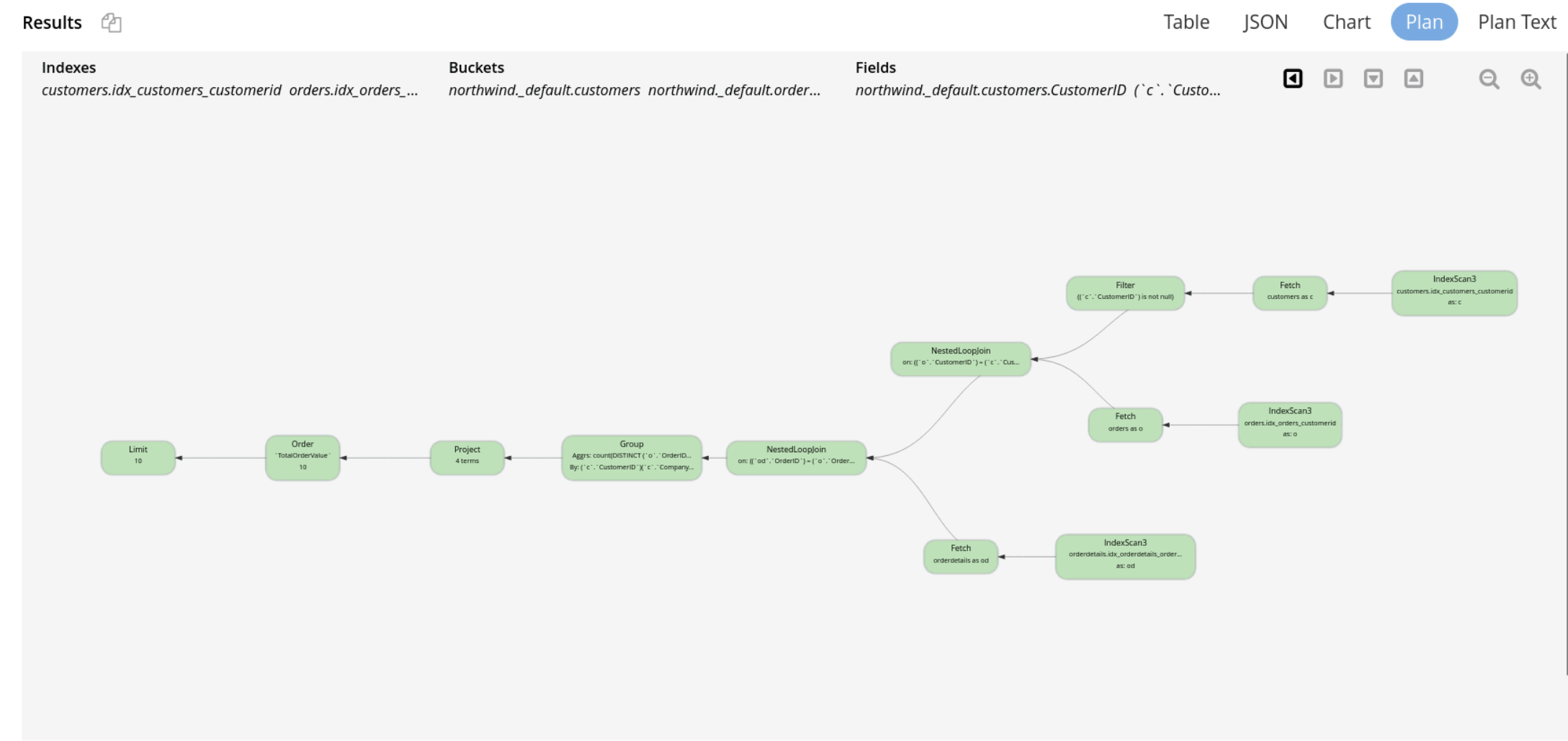
Wybierz zapytanie z zadania 3 albo 5 i uruchom je z EXPLAIN (albo za pomocą przycisku w menu).

Przykład:

```
EXPLAIN
SELECT ...
```

Zapytanie 5B:

```
select c.CustomerID,
       c.CompanyName,
       count(distinct o.OrderID) as TotalOrderCount,
       sum(od.UnitPrice * od.Quantity * (1 - ifmissingornull(od.Discount, 0))) TotalOrderValue
from `northwind`._default.customers as c
     join `northwind`._default.orders as o
       on o.CustomerID = c.CustomerID
     join `northwind`._default.orderdetails as od
       on od.OrderID = o.OrderID
group by c.CustomerID, c.CompanyName
order by TotalOrderValue desc
limit 10;
```



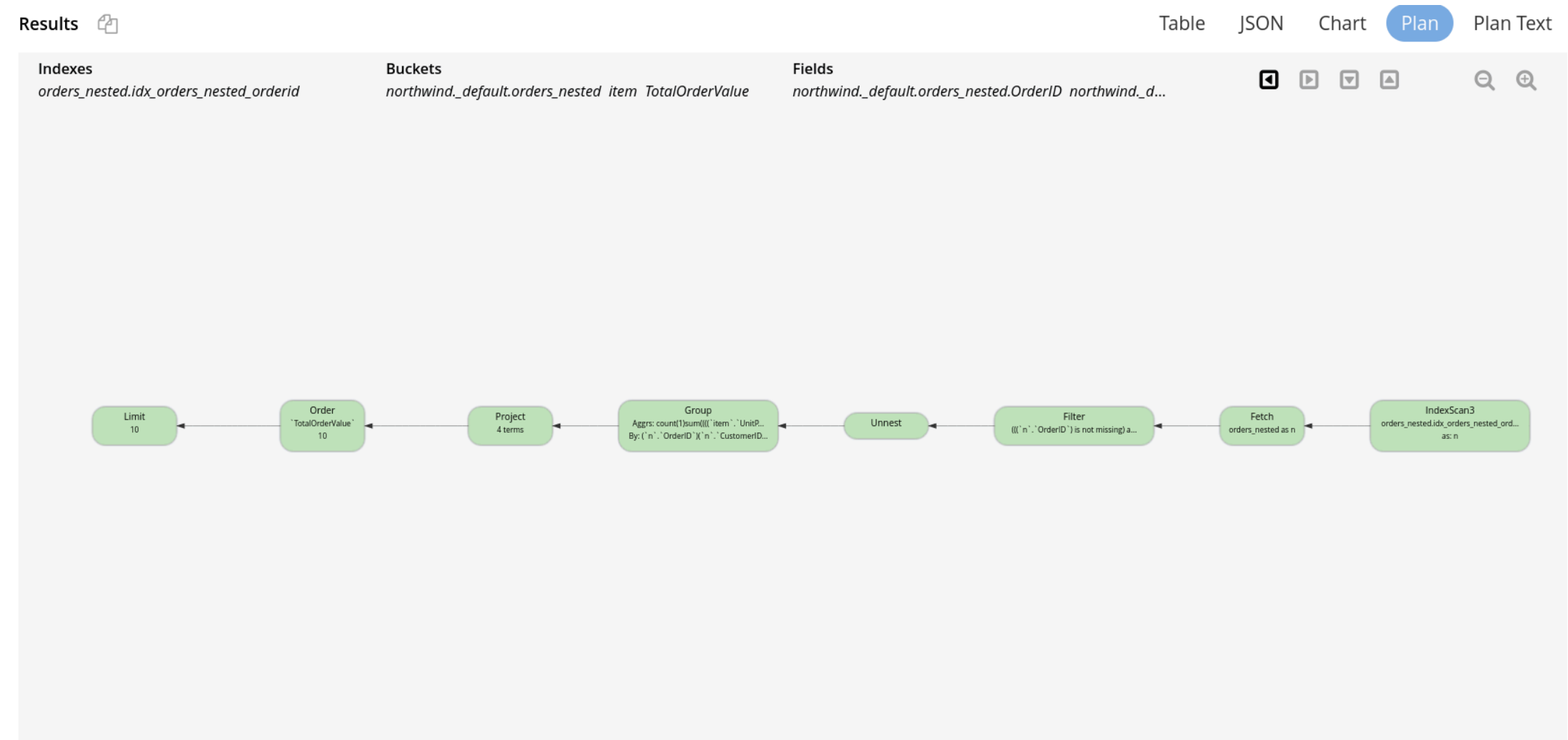
Wizualizacja 19: Plan wykonania dla zapytania z JOIN

Część B – plan dla zapytania z UNNEST

Wybierz zapytanie z zadania 4 i uruchom je z EXPLAIN.

Zapytanie 4C:

```
select n.OrderID,
       n.CustomerID,
       sum(item.UnitPrice * item.Quantity * (1 - ifmissingornull(item.Discount, 0))) as TotalOrderValue,
       count(1) as ItemsCount
from `northwind`._default.orders_nested as n
  unnest n.items as item
where n.OrderID is not missing
group by n.OrderID, n.CustomerID
order by TotalOrderValue desc
limit 10;
```



Wizualizacja 20: Plan wykonania dla zapytania z UNNEST

Część C – porównanie

Porównaj oba plany na poziomie ogólnym.

Nie opisuj całego planu. Wystarczy wskazać, z jakich indeksów korzysta zapytanie, czy pojawia się JOIN albo UNNEST, oraz gdzie widać agregację i sortowanie.

Wskaż elementy typu:

- IndexScan,
- Fetch,
- NestedLoopJoin,
- Unnest,
- Group,
- Order,
- Limit.

Porównanie:

Zapytanie z join korzysta z 3 indeksów (`idx_customers_customerid`, `idx_orders_customerid`, `idx_orderdetails_orderid`) w celu IndexScan na kolekcjach `customers`, `orders` i `orderdetails`, a następnie łączy dane za pomocą `NestedLoopJoin`. Jest to typowe podejście dla modelu relacyjnego. Zapytanie z `unnest` korzysta z tylko jednego indeksu (`idx_orders_nested_orderid`) do IndexScan na kolekcji `orders_nested`, a następnie rozбивa tablicę `items` za pomocą `unnest`. Główną różnicą jest liczba `fetch`, podejście z `join` wymusza pobranie danych z trzech kolekcji, podczas gdy `unnest` operuje na jednej kolekcji z zagnieżdżonymi danymi, co może być bardziej efektywne, ale wymaga innego modelowania danych. Pozostałe elementy planu, takie jak `Group`, `Order`, `Project` i `Limit`, są takie same z dokładnością do zwracanych kolumn i warunków.

W komentarzu końcowym napisz

Odpowiedz w kilku zdaniach:

- Co było największą różnicą między Couchbase a klasyczną bazą relacyjną?

Największą różnicą jest model danych oparty na dokumentach JSON, który pozwala na elastyczne przechowywanie danych bez sztywnej struktury tabel, oraz konieczność tworzenia indeksów, aby zapytania mogły być wykonywane. Ponadto, Couchbase oferuje funkcje specyficzne dla pracy z dokumentami, takie jak UNNEST do rozbijania tablic zagnieżdżonych w dokumentach oraz IFMISSINGORNULL do obsługi brakujących danych, co różni się od tradycyjnego modelu relacyjnego.

- Dlaczego indeksy są tak ważne w Couchbase?

Indeksy są kluczowe w Couchbase, ponieważ bez nich zapytania nie mogą być wykonane. Couchbase wymaga indeksów, aby szybko odnaleźć dokumenty spełniające warunki zapytania. Bez indeksów baza musiałaby skanować wszystkie dokumenty w kolekcji, co jest nieefektywne i może prowadzić do długiego czasu odpowiedzi.

- Co pokazało porównanie JOIN i UNNEST?

Porównanie JOIN i UNNEST pokazało, że oba podejścia mogą być używane do osiągnięcia tego samego celu biznesowego, ale różnią się sposobem modelowania danych i wykonywania zapytań. JOIN jest bardziej zbliżony do tradycyjnego modelu relacyjnego, podczas gdy UNNEST pozwala na pracę z danymi zagnieżdżonymi w dokumentach JSON, co może być bardziej efektywne w niektórych przypadkach, ale wymaga innego podejścia do projektowania bazy danych.

- Czy dokumentowy model danych wyklucza analizę i raportowanie?

Nie, ale praca z dokumentami JSON wymaga innego podejścia do modelowania danych i zapytań, zwłaszcza gdy dane są zagnieżdżone.

- Gdybyś projektował system zamówień, kiedy rozważyłbyś zagnieżdżenie pozycji zamówienia w dokumencie zamówienia?

W przypadku, gdy zamówienie składa się z wielu elementów, które są ściśle powiązane i często pobierane razem. Jest to wtedy wygodne, bo dokument zamówienia zawiera wszystkie informacje o zamówieniu, a UNNEST pozwala łatwo rozbić pozycje na osobne rekordy do analizy.

Zadanie dodatkowe dla chętnych

Materializacja KPI klienta

Utwórz kolekcję customer_kpis, a następnie zapisz do niej gotowe dokumenty z metrykami klienta:

- CustomerID,
- CompanyName,
- Revenue,
- OrdersCount.

Następnie wykonaj zapytanie raportowe na kolekcji customer_kpis.

Uwaga: wskazówki do rozwiązania zadania znajdziesz w ściądze.

Rozwiązanie

Stworzenie kolekcji

CREATE COLLECTION `northwind`._default.customer_kpis;

Wynik:

Query Editor

< history (52/52) >

context

▼

1

create collection `northwind`._default.customer_kpis

Execute

Explain

External Query Advisor

✔ success

just now | 28.7ms

format

Results

Table

JSON

Chart

Plan

Plan Text

1

{

2

"results": []

3

}

Wizualizacja 21: Stworzenie kolekcji

Zapytanie do materializacji KPI

```
UPSERT INTO `northwind`.`_default.customer_kpis` (KEY k, VALUE v)
WITH data AS (
  SELECT c.CustomerID AS k,
         c.CompanyName,
         COUNT(DISTINCT o.OrderID) AS OrdersCount,
         SUM(od.UnitPrice * od.Quantity * (1 - IFMISSINGORNULL(od.Discount, 0))) AS Revenue
  FROM `northwind`.`_default.customers` AS c
  JOIN `northwind`.`_default.orders` AS o ON o.CustomerID = c.CustomerID
  JOIN `northwind`.`_default.orderdetails` AS od ON od.OrderID = o.OrderID
  GROUP BY c.CustomerID, c.CompanyName
)
SELECT k,
       {
         "CompanyName": CompanyName,
         "OrdersCount": OrdersCount,
         "Revenue": Revenue
       } AS v
FROM data;
```

Wynik:

Query Editor

< history (56/56) >

context

1 UPSERT INTO `northwind`.`\_default.customer\_kpis` (KEY k, VALUE v)

2 WITH data AS (

3 SELECT c.CustomerID AS k,

4 c.CompanyName,

5 COUNT(DISTINCT o.OrderID) AS OrdersCount,

6 SUM(od.UnitPrice * od.Quantity * (1 - IFMISSINGORNULL(od.Discount, 0))) AS

7 FROM `northwind`.`\_default.customers` AS c

8 JOIN `northwind`.`\_default.orders` AS o ON o.CustomerID = c.CustomerID

9 JOIN `northwind`.`\_default.orderdetails` AS od ON od.OrderID = o.OrderID

10 GROUP BY c.CustomerID, c.CompanyName

11)

12 SELECT k,

13 {

14 "CompanyName": CompanyName,

15 "OrdersCount": OrdersCount,

Execute

Explain

External Query Advisor

✓ success just now | 182ms | 89 mutations

format

Results

Table

JSON

Chart

Plan

Plan Text

1 {

2 "results": []

3 }

Wizualizacja 22: Zapytanie do materializacji KPI

Stworzenie indeksu na CustomerID

```
CREATE INDEX idx_customerkpis_customerid
ON `northwind`.`_default.customer_kpis` (CustomerID);
```

Wynik:

Query Editor

< history (59/59) >

context

1 CREATE INDEX idx_customerkpis_customerid

2 ON `northwind`.`\_default.customer\_kpis` (CustomerID);

Execute

Explain

External Query Advisor

✔ success just now | 2.3s

format

Results

Table JSON Chart Plan Plan Text

1 {

2 "results": []

3 }

Wizualizacja 23: Stworzenie indeksu

Zapytanie raportowe na kolekcji customer_kpis

```
select * from `northwind`.`_default.customer_kpis`
order by Revenue desc
```

Wynik:

Query Editor

< history (63/63) >

context

1 select * from `northwind`.`\_default.customer\_kpis`

2 order by Revenue desc

Execute

Explain

External Query Advisor

✓ success just now | 2.1ms | 89 docs

format

Results

Table

JSON

Chart

Plan

Plan Text

CompanyName	OrdersCount	Revenue
QUICK-Stop	28	110277.3048835089
Ernst Handel	30	104874.97835350108
Save-a-lot Markets	31	104361.94986812412
Rattlesnake Canyon Grocery	18	51097.80048773782
Hungry Owl All-Night Grocers	19	49979.90494634077
Hanari Carnes	14	32841.36997813582
Königlich Essen	14	30908.383968844264
Folk och fä HB	19	29567.562475292383
Mère Paillard	13	28872.189950353655
White Clover Markets	14	27363.60497214645
Frankenversand	15	26656.55946504615
Queen Cozinha	13	25717.497473042273
Berglunds snabbköp	18	24927.577488892155
Suprêmes délices	12	24088.7799842298
Piccolo und mehr	10	23128.8599542819
HILARION-Abastos	18	22768.763985675276
Bon app'	17	21963.252474311183
Bottom-Dollar Markets	14	20801.599995730816
Richter Supermarkt	10	19343.778986605037

W komentarzu napisz:

- czym różni się liczenie raportu „w locie” od czytania gotowej kolekcji KPI,

Przy liczeniu w locie, zapytanie musi przetworzyć wszystkie dane źródłowe za każdym razem, gdy jest uruchamiane, co może być czasochłonne, zwłaszcza przy dużych zbiorach danych. W przypadku gotowej kolekcji KPI, metryki są już obliczone i przechowywane, więc zapytanie raportowe jest szybkie, ponieważ odczytuje gotowe wartości, ale może być zawierać nieaktualne dane, jeśli uległy one zmianie od czasu ostatniej aktualizacji KPI.

- kiedy takie podejście może być użyteczne,

Gdy dane źródłowe są duże i kosztowne do przetworzenia, a raporty są często uruchamiane, co wymaga szybkiego czasu odpowiedzi.

- jakie jest ryzyko materializowania wyników, jeśli dane źródłowe się zmieniają.

Wyniki mogą być nieaktualne, bo kolekcja KPI nie jest automatycznie aktualizowana przy każdej zmianie danych źródłowych.